From Optimal Control to Diffusion Policy

A Trajectory-Optimization → TVLQR → Behavioural-Cloning Pipeline for Double-Pendulum Swing-Up

Group Project TN2 — Technical Documentation

June 26, 2026

Contents

1 Introduction

The goal of this project is to compare the performance of 2 different learning- based control methods. The first method is a behavioural cloning approach, where we use a dataset of expert demonstrations to train a policy that mimics the expert's actions. The second method is a diffusion policy approach, where we use a diffusion model to learn a policy that can generate actions based on the current state of the system. We will evaluate the performance of both methods in the **double pendulum swingup task** and the **stacking problem**.

1.1 Behavioural Cloning

1.2 Diffusion

In order to understand the diffusion policy approach, we first need to understand the concept of diffusion models. Diffusion models are a class of generative models that learn to generate structured data (in our case, action sequences) by iteratively denoising a sample of noise.

Parameter Setup

For the diffusion model, we first need to define the time parameters of the noising and denoising process.

- **Time Horizon** T is the number of time steps in the diffusion process.
- β **Schedule** β_t (with t $\in \{1, 2, \ldots, T\}$) is a set of hyperparameters that control how quickly the noise is added to the data during the forward diffusion process. It is a monotonic increasing sequence of values that determine the variance of the noise
- **Noise Schedule** $\alpha_t = 1 - \beta_t$ is a set of hyperparameters that control how quickly the noise is removed.
- **Cumulative Noise Schedule** $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$

So we can define the forward diffusion process as follows:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \tag{1}$$

where x_0 is the original data (action sequence) and x_t is the noisy version of the data at time step t. So now we need to define a procedure to undo the noise in T steps. The reverse diffusion process is defined as follows:

$$x_0 = \frac{1}{\sqrt{\bar{\alpha}_t}} \cdot x_t - \frac{\sqrt{1 - \bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}} \cdot \epsilon(x_t, t), \quad \epsilon(x_t, t) \sim \mathcal{N}(0, I) \tag{2}$$

However, we do not have access to the true noise ϵ that was added during the forward diffusion process. Instead, we train a neural network $\epsilon_\theta(x_t, t)$ to predict the noise given the noisy data x_t and the time step t.

Algorithm 1 Diffusion Model Training

Require: Training dataset $\mathcal{D}$, diffusion steps T

Initialize network parameters θ
repeat
  for all $x_0 \in \mathcal{D}$ **do**
    Sample timestep $t \sim \text{Uniform}(\{1, \dots, T\})$
    Sample noise $\epsilon \sim \mathcal{N}(0, I)$
    Construct noisy sample

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon$$

    Predict noise

$$\hat{\epsilon} = \epsilon_\theta(x_t, t, c)$$

    Compute loss

$$\mathcal{L} = \|\hat{\epsilon} - \epsilon\|_2^2$$

  end for
  Update θ using the average batch loss.
until convergence

Algorithm 2 Reverse Diffusion Sampling

Require: Trained network ϵ_θ, conditioning vector c

Sample

$$x_T \sim \mathcal{N}(0, I)$$

for $t = T, T-1, \dots, 2$ **do**
  Predict noise

$$\hat{\epsilon} = \epsilon_\theta(x_t, t, c)$$

  Compute the posterior mean

$$\hat{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \hat{\epsilon} \right)$$

  Sample

$$\epsilon \sim \mathcal{N}(0, I)$$

  Update

$$x_{t-1} = \hat{x}_{t-1} + \sigma_t \epsilon$$

  where $\sigma_t = \sqrt{\bar{\beta}_t}$
end for
Compute the final denoised sample

$$x_0 = \frac{1}{\sqrt{\alpha_1}} \left(x_1 - \frac{\beta_1}{\sqrt{1 - \bar{\alpha}_1}} \epsilon_\theta(x_1, 1, c) \right)$$

return x_0

1.3 Generating Dataset

Both learning methods are trained by imitation, so they are only ever as good as the expert demonstrations they are shown. We therefore generate the dataset with a classical optimal-control pipeline: we (i) define a physics model of the double pendulum, (ii) solve a trajectory-

optimization problem for a nominal swing-up, (iii) wrap that nominal in a time-varying LQR (TVLQR) feedback controller, and (iv) roll the controller out *inside the simulator* to harvest dynamically consistent state–action pairs. The result is a set of expert trajectories that both the behavioural-cloning and diffusion-policy models clone.

1.3.1 Dynamics Model

The state is $x = [q_1, q_2, \dot{q}_1, \dot{q}_2]^\top$, where q_1 is the shoulder and q_2 the elbow angle, and the control $u = [u_1, u_2]^\top$ collects the two joint torques. We use $q_1 = 0$ for the stable hanging-down configuration and $q_1 = \pi$ for the upright goal. The planar two-link equations of motion are

$$M(q)\,\ddot{q} = u + G(q) - C(q,\dot{q})\,\dot{q} - b\,\dot{q} - f\,\tanh(\dot{q}/\epsilon)\,, \tag{3}$$

with mass matrix $M(q)$, Coriolis/centrifugal matrix $C(q,\dot{q})$, gravity vector $G(q)$, viscous damping b and Coulomb friction f. The non-smooth dry friction $\operatorname{sign}(\dot{q})$ is replaced by a smooth $\tanh(\dot{q}/\epsilon)$ ($\epsilon = 0.05$ rad/s) so the term is differentiable for the gradient-based solver. Crucially, every physical parameter (masses, lengths, link inertias, damping and friction) is read *directly* from the MuJoCo model file `dp.xml`, so the planner and the simulator used for deployment describe the exact same system and cannot silently drift apart.

1.3.2 Trajectory Optimization

The nominal swing-up is found by direct trapezoidal collocation, transcribed into a nonlinear program (NLP) and solved with IPOPT. Discretizing the horizon into $N+1$ knots, the decision vector stacks every state and control, $z = [x_0, \dots, x_N, u_0, \dots, u_N]$, and we minimize the quadratic cost

$$J(z) = \tfrac{1}{2}\sum_{k=0}^{N-1}\Big[(x_k - x_g)^\top Q\,(x_k - x_g) + u_k^\top R\,u_k\Big] + (x_N - x_g)^\top Q_f\,(x_N - x_g), \tag{4}$$

subject to the trapezoidal collocation (dynamics) defects, the boundary conditions, and the actuator torque limits:

$$x_{k+1} - x_k - \tfrac{\Delta t}{2}\big(f(x_k,u_k) + f(x_{k+1},u_{k+1})\big) = 0, \quad k = 0,\dots,N-1, \tag{5}$$

$$x_0 = x_{\text{init}}, \quad x_N = x_g = [\pi, 0, 0, 0]^\top, \tag{6}$$

$$|u_k| \le \tau_{\max}. \tag{7}$$

The objective gradient and the constraint Jacobian are obtained by automatic differentiation (JAX), and the problem is compiled once and reused across solves. The weighting matrices Q, R trade off tracking accuracy against control effort; sweeping them, together with the peak-torque limit and the initial condition, produces a diverse family of nominal swing-ups rather than a single trajectory.

1.3.3 TVLQR Tracking and Closed-Loop Rollout

A nominal trajectory (x^*, u^*) on its own is open-loop and brittle: applied verbatim, the smallest model mismatch or numerical drift sends this chaotic system off course. We therefore linearize the dynamics about the nominal and compute time-varying LQR feedback gains K_t, yielding the tracking law

$$u_t = u_t^* - K_t\,(x_t - x_t^*). \tag{8}$$

A nearest-neighbour rule re-locks the controller onto the closest reference state (and blends the nearby gains) whenever the deviation exceeds a threshold, giving it a measure of recovery. This controller is rolled out inside the *actual* MuJoCo simulator, and at every control step (control period $\Delta t = 0.05$ s, i.e. 20 Hz) we log the observed state together with the commanded torque.

Because each sample is produced by stepping the same simulator on which the policy is later evaluated, the dataset is dynamically valid by construction; cloning it cannot produce a policy that is "correct" for a physics model it never runs on.

1.3.4 DAgger-Style Coverage

A policy trained only on the nominal states suffers from compounding error: its own small prediction errors push it into off-nominal states it was never shown, and the errors snowball. To cover this recovery distribution, during the swing-up phase we perturb the *applied* torque with Gaussian noise ($\sigma = 0.15\,\tau_{\max}$) while still logging the *clean* commanded torque as the supervised target. Each sample then encodes "from this (possibly off-nominal) state, the expert would command this torque"—a DAgger-style relabelling that teaches recovery. The noise latches off once the system first reaches upright, leaving the fragile balancing phase undisturbed so the rollout still succeeds.

1.3.5 Dataset Composition

The data is gathered in two regimes:

- **Swing-up rollouts** start near hanging-down (angles in ± 0.6 rad, velocities in ± 1.0 rad/s) and run for up to 200 control steps. One in four is run noise-free to retain clean nominal behaviour.
- **Upright-hold rollouts** start *perturbed* about the upright equilibrium ($q_1 = \pi \pm 0.25$ rad, etc.) and are short and noise-free. Their purpose is to record the stabilizing deviation→corrective-torque map, without which the policy only ever sees the exact fixed point and never learns to balance.

Only rollouts that actually reach and hold the upright are kept. This yields approximately 600 trajectories—predominantly swing-up, with the remainder holding. Each trajectory is stored as an HDF5 group containing `states` $\in \mathbb{R}^{T\times 4}$ and `actions` $\in \mathbb{R}^{T\times 2}$ in `results/expert_trajectories.h5`, the common format consumed by both the behavioural-cloning and diffusion-policy training scripts.

Table 1: Key dataset-generation parameters.

Parameter	**Symbol**	**Value**
State / action dimension	n_x, n_u	4, 2
Upright goal state	x_g	$[\pi, 0, 0, 0]^\top$
Control period (rate)	Δt	0.05 s (20 Hz)
Friction smoothing	ϵ	0.05 rad/s
Swing-up exploration noise	σ	$0.15\,\tau_{\max}$
Swing-up rollout length	—	200 steps
Approx. trajectories kept	—	$\sim$600

1.4 Diffusion Policy Model

We follow the *Diffusion Policy* formulation of Chi et al. (2023): instead of regressing a single action from the current state, the network models the conditional distribution of a short *action chunk* given a window of recent observations, and samples from it by iterative denoising. The forward and reverse processes are exactly those of Section 1.2 above; what follows is the concrete realization for the double pendulum, with particular attention to the input representation, the normalization, and the handful of engineering choices that were necessary before the policy could actually swing up *and* balance.

1.4.1 Conditioning and Action Representation

The diffusion happens over an action chunk $a^{1:H}$ of $H = 10$ future torque pairs; the network is *conditioned* on a context vector c that is left clean (never noised). The context is built from the last $K = 4$ observed states plus the fixed goal. Two representation choices proved important:

Angular features. Feeding raw angles is harmful because q wraps at $\pm\pi$: two physically identical configurations can be numerically far apart, and the discontinuity sits right where the swing-up passes. We therefore map every state through a wrap-free angular feature transform

$$\phi(x) = \big[\sin q_1,\ \cos q_1,\ \sin q_2,\ \cos q_2,\ \tilde{\dot{q}}_1,\ \tilde{\dot{q}}_2\big] \in \mathbb{R}^6. \tag{9}$$

Min–max normalization. Velocities and torques live on very different scales from the unit-circle angle features, so both are min–max scaled into $[-1, 1]$,

$$\tilde{\dot{q}} = 2\,\frac{\dot{q} - \dot{q}_{\min}}{\dot{q}_{\max} - \dot{q}_{\min}} - 1, \qquad \tilde{a} = 2\,\frac{a - a_{\min}}{a_{\max} - a_{\min}} - 1, \tag{10}$$

with the per-dimension $(\min, \max)$ statistics computed once and *saved*, so the network output can be de-normalized back to physical torques at deployment. The diffusion target is the normalized action chunk $\tilde{a}^{1:H}$.

The conditioning vector concatenates the K feature-mapped history states with the goal features,

$$c = \big[\phi(x_{i-K+1}),\ \ldots,\ \phi(x_i),\ \phi(x_g)\big] \in \mathbb{R}^{(K+1)\cdot 6} = \mathbb{R}^{30}. \tag{11}$$

Including the velocity-carrying history (rather than a single state) gives the policy the information it needs to act on this second-order system. The goal token leaves the architecture *able*, in principle, to be retargeted to other set-points, but every trajectory in our dataset shares the single upright goal $x_g = [\pi, 0, 0, 0]^\top$; the goal input is therefore effectively constant, and the network never actually learns to condition on it. Genuine retargeting to a different goal would require demonstrations at that goal. Windows are zero-padded at the trajectory ends by repeating the first state / last action.

1.4.2 Noise-Prediction Network

The network $\epsilon_\theta(\tilde{a}_t^{1:H}, t, c)$ predicts the noise added to the action chunk and is trained with the standard ϵ-MSE objective. We implement two interchangeable backbones behind a common `forward`(a_{noisy}, t, c) contract:

- **MLP** — flattens the chunk to $H \cdot n_u$, concatenates a learned embedding of the (normalized) diffusion timestep and of c, and passes it through a SiLU MLP. It ignores the temporal structure of the chunk.
- **Trajectory Transformer** (default) — treats the H actions as a token sequence with learned positional embeddings, and *prepends two context tokens* (one for the diffusion timestep, one

for c) so every action token can attend to them. It uses pre-norm encoder layers with GELU activations; only the H action positions are projected back to torques. This preserves the temporal correlation within a chunk that the MLP discards.

The diffusion timestep enters as a sinusoidal embedding (transformer) or a small learned embedding (MLP), with t normalized to $[0, 1]$ by dividing by T.

1.4.3 Noise Schedule and Terminal SNR

We use a linear β schedule, $\beta_t \in [3 \times 10^{-4}, 0.1]$, over the $T = 20$ diffusion steps. Sampling starts from pure Gaussian noise, so the inference prior only matches the forward marginal $q(x_T \mid x_0)$ to the extent that the terminal signal-to-noise ratio is small — ideally the zero-terminal-SNR condition $\bar{\alpha}_T \approx 0$ (Lin et al. 2023). The upper end of the schedule is therefore deliberately large: an earlier, milder `end_beta` left far more of the action signal intact at x_T, widening the train/inference mismatch, and raising it reduces that residual.

How far this goes depends on *both* `end_beta` and the number of steps, since $\bar{\alpha}_T = \prod_{t=1}^{T}(1-\beta_t)$ keeps shrinking as more steps accumulate noise. At the short horizons we sample with, the condition is approached but not reached: at $T = 20$ this schedule gives $\bar{\alpha}_T \approx 0.35$, so roughly 60% of the signal amplitude ($\sqrt{\bar{\alpha}_T}$) is still present at x_T, and only at much larger T (e.g. $T \approx 100$, where $\bar{\alpha}_T \approx 0.006$) is the terminal SNR effectively zero. We therefore do *not* claim a true zero-terminal-SNR prior; the enlarged `end_beta` shrinks the prior mismatch enough to denoise reliably at the low, fast-to-sample T that the closed-loop controller depends on.

1.4.4 Training Tricks

- **Trajectory-level split, train-only stats.** The data is split 70/15/15 into train/val/test *by trajectory* before any window slicing, so no two windows of the same trajectory straddle the boundary. Normalization statistics are computed on the *train fold only* and reused for val/test; letting the held-out folds normalize themselves would leak their distribution into the inputs.

- **EMA weights.** We keep an exponential moving average (decay = 0.999) of the network weights and use the EMA copy for all inference. The EMA is a much smoother estimate of the learned map; near the *unstable* upright equilibrium this matters concretely, because a jittery gain perturbs the torque and topples the pendulum. The EMA model is what gets saved to the checkpoint.

- **Optimizer / schedule.** Adam with a per-step cosine-annealing learning-rate schedule, plus mixed-precision (AMP) training on GPU.

1.4.5 Inference and Control Tricks

At control time the policy denoises an action chunk via DDPM ancestral sampling (reverse diffusion over the T steps), conditioned on the current history and goal, then de-normalizes the result back to physical torques. Two choices are what make the closed loop stable rather than merely plausible:

- **Deterministic (noise-free) sampling for control.** The ancestral noise injected at each reverse step is dropped, so the sampler returns the posterior mean. This sharply lowers action variance — essential when *stabilizing* the unstable upright, where stray sampling noise on the torque is enough to drop the pendulum. (Stochastic sampling is kept available for when sample diversity, rather than control, is the goal.)

- **Short execution horizon.** Although the network predicts $H = 10$ actions, only the leading n_{exec} are executed before re-planning (receding-horizon action chunking). In practice executing

Table 2: Diffusion-policy model and training hyperparameters (defaults).

Parameter	**Symbol**	**Value**
Observation history	K	4 states
Feature dimension	—	6 per state
Action prediction horizon	H	10
Conditioning dimension	—	30
Diffusion steps	T	20
β schedule	β_t	linear [3×10^{-4}, 0.1]
EMA decay	—	0.999
Optimizer / LR	—	Adam, 10^{-4} (cosine)
Batch size / epochs	—	256 / 400
Default backbone	—	Transformer (d=128, 4 heads, 2 layers)

a single action per re-plan is what reliably swings up; committing to longer open-loop chunks lets the chaotic dynamics diverge from the plan.

1.5 Comparison